

IT University of Copenhagen

Building the Democracy Stack

What Software Architecture Decisions Matter
for Democratic Digital Infrastructure?

Author Leo William Engholm Sakharov

Email leos@itu.dk

Programme MSc Software Design

Course Research Project (7.5 ECTS)

STADS code KIREPRO1PE

Supervisor Konstantinos Manikas

Date 15 May 2026

Abstract

Civic technology platforms have been building digital democratic infrastructure for over a decade — software for participatory budgeting, policy proposals, and citizen consultations. European policy is starting to call for the same thing, under names like the Digital Services Act, the European Digital Identity framework, IDEA’s “democracy stack”, and CEPS’s case for European digital public infrastructure. But policy talks about what this infrastructure should do, not how it should be built. The platforms themselves have rarely been analysed through software architecture and software ecosystem (SECO) methods. This paper proposes a small analytical framework for thinking about software architecture decisions in democratic digital infrastructure. The framework has three parts: four quality attributes that connect architecture to democratic capacities (modifiability, interoperability, configurability, and transparency and auditability); Christensen et al.’s three-structures lens, where ecosystem health depends on the fit between organisational, business, and software structures; and a set of architectural trade-offs that follow from the four attributes. The paper also develops *fork-divergence shape*, a way to read niche creation from the shape of a platform’s fork population.

The framework is applied to a comparative case study of Decidim and Consul, two civic tech platforms with shared origins in Spain’s 15M movement and opposite architectural paths. The application uses architectural recovery, evolutionary analysis, and a fork-divergence study across all public forks of both platforms (1,030 for Consul, 452 for Decidim). Decidim’s modular software fits its coordinating Association and consultancy-based funding. Consul’s monolithic software needs coordination that its small Foundation and shoestring funding cannot provide. The fork-divergence shapes invert at the originator: Madrid (Consul) sits in an abandoned 12,725-commit silo, Barcelona (Decidim) tracks upstream with zero local commits. On each platform the diagnostic reads a different niche — deployment fragmentation for fork-based Consul, contributor flow for plugin-based Decidim — depending on the platform’s extension model. Quality attributes where Decidim and Consul behave similarly — security, accessibility, sovereignty — are left for future work.

Contents

1	Introduction	3
2	Background	3
2.1	Software Architecture Quality Attributes	4
2.2	Software Ecosystems	4
2.3	Architecture as Governance	5
3	Method	5
4	Framework	8
4.1	The Three Components	8
4.1.1	Quality Attributes Mapped to Democratic Capacities	8
4.1.2	The Three-Structures Lens	8
4.1.3	Architectural Trade-offs	9
4.2	Fork-Divergence Shape: A Niche-Creation Diagnostic	10
5	Application: Decidim and Consul	11
5.1	Decidim	11
5.2	Consul	16
5.3	Cross-case Synthesis	19
6	Discussion	21
6.1	Architecture as Governance in Practice	21
6.2	What SECO Research Gains	21
6.3	Implications for the Democracy Stack	22
6.4	Limitations	23
7	Conclusion	23
	Use of Generative AI	24

1 Introduction

European policy is starting to call for digital infrastructure for democracy — the Digital Services Act [European Union, 2022], the European Digital Identity framework [European Union, 2024], IDEA’s “democracy stack” [International IDEA, 2026], and CEPS’s case for European Digital Public Infrastructure [CEPS, 2025]. These proposals say what the infrastructure should do, not how it should be built. Civic technology platforms have been building this kind of software for over a decade — Decidim across hundreds of institutions in 32 countries [Barandiaran et al., 2024], Consul across over 250 cities and organisations [NTARI, 2025] — yet how their architectures shaped their ecosystems remains under-analysed.

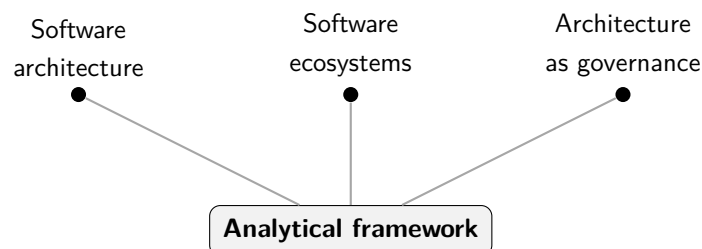
This paper asks: *what software architecture decisions matter for democratic digital infrastructure, and how can they be analysed through software architecture and software ecosystem methods?*

The methods exist across software architecture, SECO, STS/platform studies, and fork-mining research (Section 2), but have rarely been combined to analyse civic tech: Manikas [2016]’s longitudinal review of 231 SECO studies (2007–2014) is dominated by industrial open-source and proprietary platforms; civic participation platforms do not appear as a recognised category in his classification.

We propose a small analytical framework combining four quality attributes (modifiability, interoperability, configurability, transparency and auditability), Christensen et al. [2014]’s three-structures lens, and a set of trade-offs. We apply it to Decidim and Consul — two platforms with shared origins in Spain’s 15M movement (2015–2016) but opposite architectural paths (Decidim modular, 27 Ruby engines + coordinating Association; Consul monolithic, deployers fork + small Foundation coordinates from the outside) — using architectural recovery, commit history analysis, and a fork-divergence study across 1,030 Consul and 452 Decidim public forks. From the application we surface *fork-divergence shape*, a population-shape niche-creation diagnostic whose reading depends on the extension model. Decidim and Consul are first worked cases, not a full validation across the field.

2 Background

Reasoning about democratic digital infrastructure needs three literatures: software architecture (for quality attributes and trade-offs), software ecosystems (for the organisational, business, and technical structures around a platform), and architecture-as-governance (for how software architectures make some forms of action, coordination, and control easier than others).



2.1 Software Architecture Quality Attributes

Bass et al. [2021, p. 4] call software architecture “the set of structures needed to reason about a system.” Architecture decisions affect a system’s properties over its lifetime through quality attributes: measurable properties like modifiability, performance, security, and so on. Quality attributes are useful here because they let us evaluate architecture without reducing it to implementation details. They allow us to ask not only what a platform contains, but what kinds of change, inspection, integration, and adaptation its structure makes easier or harder.

In commercial software, quality attributes are usually optimised for business value. For democratic digital infrastructure the goal is different: which architectural properties support democratic participation, public legitimacy, and institutional adaptability?

We focus on four quality attributes.

Modifiability. How easily the system can be changed. Civic tech platforms have to adapt to communities with different rules, languages, and participatory traditions; modifiable platforms can do this without forcing each deployer to maintain a separate fork.

Interoperability. How well the system exchanges data and services with others. This matters because no single platform will serve all democratic functions, so platforms need to compose.

Configurability. The platform’s support for configuring varied participation processes (proposals, voting, deliberation, accountability) without changing the source code. PolicyKit [Zhang et al., 2020] prototyped programmable governance for online communities, a close analogue of this property at the governance-rule layer.

Transparency and auditability. How much of the platform’s operation can be inspected by relevant observers (transparency), and how reliably those observations can be used to verify and contest outcomes (auditability). Democratic legitimacy depends partly on both, although technical auditability does not by itself guarantee that citizens can meaningfully use these mechanisms in practice.

Other quality attributes (security, accessibility, sovereignty, performance) matter for democratic infrastructure too. We do not treat them as primary axes here and return to this scoping in Section 6.

2.2 Software Ecosystems

SECO research broadens the unit of analysis from the software to the wider system around it. Manikas and Hansen [2013] define a software ecosystem (SECO) as “the interaction of a set of actors on top of a common technological platform, which results in a number of software solutions or services.” The concept extends beyond a single product to the organisations, developers, and users that develop around a shared platform.

Christensen et al. [2014] propose that ecosystems can be analysed through three interacting structures: an *organisational* structure (who participates, in what roles, how they coordinate), a *business* structure (who pays, who captures value, what incentives flow), and a *software* structure (the platform architecture, extension mechanisms, APIs). Their argument is that ecosystems thrive when these three structures fit together and struggle when they do not. We use this lens in Section 5 to compare Decidim and Consul.

We also draw on Iansiti and Levien [2004]’s ecosystem actor classification (keystones, dominators, niche creators) and Jansen [2014]’s ecosystem health dimensions (productivity, robustness, niche creation). In this paper, these concepts are treated as analytical lenses rather than direct measurements of democratic value.

Manikas [2016]’s review of 231 SECO studies catalogues the field’s studied ecosystems in Appendix A: Android, Eclipse, Gnome, Apple AppStore, Apache, Linux, Ruby, and similar industrial platforms dominate; the few public-sector entries (Brazilian Public Software, US Department of Defense) treat government as a procurement context rather than a citizen-participation site. Civic, commons, and democratic platforms in the Decidim/Consul sense are absent from the 2007–2014 corpus and from Manikas’s classification scheme. Knutas et al. [2020] proposed applying OSSECO methods to civic tech but did not carry out the empirical work. Palacin et al. [2024] study Decidim’s modularity from a human-computer interaction perspective, but do not analyse the architecture itself as part of a software ecosystem.

2.3 Architecture as Governance

A long tradition in science and technology studies (STS) and platform studies has established that software architectures are not politically neutral: they distribute possibilities, making some forms of participation, oversight, and control easier than others [Winner, 1980, Lessig, 2006, DeNardis, 2014, Schneider, 2024]. This reading is analytical rather than deterministic — architecture does not by itself cause political outcomes. Schneider et al. [2021]’s *Modular Politics* moves toward design, proposing four principles (modularity, expressiveness, portability, interoperability) for a governance layer in online communities. Two of these principles overlap with QAs developed in this paper (modularity with modifiability, interoperability with interoperability), and we revisit this resonance in Section 4. These ideas are directly relevant to civic technology platforms, but have rarely been combined with software architecture and SECO methods.

3 Method

This paper is a comparative case study of two civic technology platforms, Decidim and Consul, following the case-study guidelines for software engineering of Runeson and Höst [2009]. The empirical work uses two methods: architectural recovery [Ducasse and Pollet, 2009, Garcia et al., 2013] for reconstructing each platform’s software structure from primary sources, and Mining Software Repositories [Hassan, 2008] for the commit-history, contributor-concentration, logical-coupling, and fork-divergence analyses. The framework in Section 4 is built from existing software architecture and SECO literature; the application in Section 5 demonstrates it on the

two cases but does not validate it across the wider civic tech field. Evaluation across more cases is left for future work.

Case selection. We selected Decidim and Consul as the comparative case for two reasons. First, both have publicly developed codebases, active deployments, and enough documentation for architectural recovery. Second, the two platforms share an origin in Spain’s 15M movement in 2015–2016 but took opposite architectural paths: Decidim went modular (27 Rails engines, gem-based extension); Consul stayed monolithic (single Rails app, fork-based extension). Both platforms emerged from Spain’s 15M movement in 2015–2016, so the era, country, and political project are held constant; the architectural divergence is the contrast we want to study. This is theoretical case selection: the cases are chosen for their analytical contrast, not statistical representativeness. Other civic tech platforms (Polis, Loomio, CitizenLab, vTaiwan, Adhocracy) are not analysed; a single two-platform comparison is enough to demonstrate the framework at this stage, but not enough to generalise across civic tech as a whole.

Architectural recovery. For each platform we recover the software structure from primary sources: codebase organisation, gemspec files, extension mechanisms, and documentation. For Decidim, a script parses all 27 of Decidim’s own gemspecs at a pinned snapshot, extracts every `add_dependency` call, classifies each as internal or external, and identifies `Decidim.register_component` and `Decidim.register_participatory_space` calls. The engine dependency graph in Section 5 is recovered from this data, not asserted from documentation. Consul’s recovery is structural: a Rails monolith with a file-override extension mechanism. The recovery captures dependency structure and extension mechanisms, not runtime behaviour or deployment-specific customisations.

Evolutionary analysis. For both platforms we extract commit metadata with `git log --no-merges`. Bot authors (Dependabot, Crowdin, Weblate, GitHub Actions, named release bots) are filtered by email and name patterns. We compute total human commits, unique authors, top-three and top-ten contributor share, Gini coefficient (a measure of contributor concentration: 0 means commits are evenly distributed across authors, 1 means one author makes them all), and the per-year commit timeline; for Decidim we also extract per-engine commit counts. We use these as indicators of ecosystem activity and concentration [Hassan, 2008, Manikas and Hansen, 2013, Jansen, 2014], not as direct measures of ecosystem health or democratic value.

Logical coupling. Following Gall et al. [1998], we measure which Decidim engines change together. For each non-merge non-bot commit we identify the engines touched and compute Jaccard similarity (commits touching both / commits touching either) for each engine pair. Mega-commits touching more than eight engines are excluded as refactor or release commits that would otherwise dominate the signal. Logical coupling is computed only for Decidim, since Consul’s monolithic structure has no engine boundary to test against. This tests whether the modular structure shows up in how commits actually fall, not only in the declared dependencies.

Fork divergence. For both platforms we use the GitHub REST API to enumerate all public forks of the upstream repository (`consuldemocracy/consuldemocracy`, formerly `consul/consul`; and `decidim/decidim`) and call the `compare` endpoint for each, returning commits ahead and behind upstream’s default branch (`master` for Consul, `develop` for Decidim), drawing on the fork-mining literature [Zhou et al., 2020, Wang et al., 2023, Businge et al., 2022]. Each fork is classified as *modified* or *unmodified* (commits ahead) and *active* or *dormant* (pushed in the last three years; this is a methodological choice rather than a fixed convention). The analysis covers public forks only and may miss private production forks. The two platforms have different extension models: Consul deployers customise by forking the application, while Decidim deployers install the gem in their own Rails application and extend through plugins. As a result, the same diagnostic reads different ecosystem niches in each case — deployment fragmentation for Consul, contributor flow for Decidim — a methodological point we return to in the cross-case synthesis and discussion.

Three-structures application. For each platform we describe the three structures from Christensen et al. [2014] — organisational, business, software — using primary documentation (Decidim Association governance documents, MetaDecidim assembly records, Consul Democracy Foundation website, OSOR case studies) and the empirical material from architectural recovery and evolutionary analysis. We then assess alignment or misalignment between the three structures. This assessment is interpretive: it asks whether the software structure fits the organisational and business reality around it, and whether the organisational and business structures can sustain the architectural costs. Specifically, alignment evidence includes whether the organisational structure has roles to use the software’s extension mechanisms, whether the business structure can sustain the architectural maintenance costs, and whether contribution flows match the paths the software allows. It does not establish causal proof.

Quality-attribute assessment. The four quality attributes are assessed qualitatively as low, moderate, or high. Table 1 gives the criteria per QA per level. The ratings are based on architectural mechanisms and evidence from the repository and documentation; they are structured judgements, not measurements.

Data sources. Primary sources are the public GitHub repositories at pinned snapshots: `decidim/decidim` at `be39c244` (2026-05-12) and `consuldemocracy/consuldemocracy` at `3c3b5c4a` (2026-05-11). Secondary sources include Barandiaran et al. [2024] on Decidim’s sociotechnical design, the OSOR case study on Consul’s European deployments [Linåker and Gates, 2025], the NTARI fork report [NTARI, 2025], and the Codegram architecture blog on early Decidim design [Codegram, 2024]. Reproducibility scripts (`scripts/` directory: gems spec parsing, commit metadata extraction, fork-divergence analysis) and findings logs are publicly available at <https://github.com/leosakharoff/democracy-stack-paper>.

Table 1: Quality-attribute rating rubric: criteria for low, moderate, and high ratings.

Quality attribute	Low	Moderate	High
Modifiability	Extension requires core code changes or a deployment fork	Plugin hooks exist; some adaptation still requires forking	Modules extend the platform; no core changes or forks needed
Interoperability	No public API or shared data formats	Partial API exposure or limited standard-format support	Stable public API; standard formats supported (e.g., ActivityPub, OpenAPI)
Configurability	Participation processes fixed in code	Some processes configurable; limited coverage	Varied processes composable via settings; no code changes
Transparency and auditability	Operations not logged beyond standard application logs	Some inspection mechanisms; not first-class architectural concerns	First-class logging, versioning, and gated public inspection

4 Framework

4.1 The Three Components

The three components below offer overlapping but complementary views of a platform: the quality attributes look at the software structure itself, the three-structures lens locates the software within its organisational and business context, and the architectural trade-offs name the choices that follow from the QA mapping.

4.1.1 Quality Attributes Mapped to Democratic Capacities

Each of the four quality attributes introduced in Section 2.1 connects an architecture choice to a democratic capacity the platform can support. Each one also pairs with an architectural cost the platform has to pay if it chooses that direction. Table 2 collects the mappings.

The mappings are analytical, not causal proofs: they identify democratic capacities each attribute can support when matched by appropriate organisational and institutional conditions, and a different corpus or normative starting point would land on different mappings. Modifiability and configurability address different audiences — developers and administrators respectively — and both matter for democratic infrastructure.

4.1.2 The Three-Structures Lens

The QAs above describe properties of the software structure alone. Christensen et al. [2014]’s three-structures framework gives us a way to think about how the software structure relates to the broader ecosystem.

The three structures (organisational, business, software) are introduced in Section 2.2.

Table 2: Quality attributes mapped to democratic capacities and the architectural costs they impose.

Quality attribute	Democratic capacity	Architectural cost
Modifiability	Pluralism: diverse communities can adapt the platform	Sustainability burden of N composable parts
Interoperability	Composability: platforms can join wider stacks	Reduced local autonomy under shared standards
Configurability	Procedural pluralism: participation can be organised in different ways	Configuration surface complexity; maintenance overhead
Transparency and auditability	Legitimacy, accountability, and contestability can be supported through inspection and verification	Audit infrastructure: logs, versioning, permissions, and API discipline

The lens locates the QAs in their organisational and business context: modifiability matters for pluralism only if contributors can use the extension mechanisms; transparency matters for legitimacy only if actors are able to inspect and act on what they see. It also names a failure mode beyond bad architecture — *misalignment* between the software structure and the organisational and business reality around it. A modular architecture without a coordinating body may produce fragmentation rather than pluralism; a modifiable platform without sustained funding may produce decay rather than adaptation.

4.1.3 Architectural Trade-offs

The four QAs cannot all be optimised without cost. Each one pairs with an architectural burden (Table 2), and choosing one direction means accepting that burden elsewhere in the ecosystem.

Three pairings are particularly visible in civic tech and form the framework’s architectural trade-offs:

Modifiability against sustainability burden. Modular architectures let many deployers — municipalities, NGOs, civic-tech consultancies — adapt the platform; the cost is maintaining many parts. Monolithic architectures can be easier to maintain centrally; the cost is that adaptation often requires forking or direct source modification unless other extension mechanisms are provided.

Interoperability against reduced autonomy. Interoperable platforms can plug into wider stacks via shared standards like ActivityPub, OAuth, or OpenAPI; the cost is that those standards constrain what the platform can change unilaterally — switching to a different data model or auth flow would break the integration. Stand-alone platforms keep that local autonomy but cannot share users, data, or features with other systems.

Configurability against maintenance overhead. Highly configurable platforms support many different participation processes without code changes; the cost is configuration surface complexity and the maintenance of options across versions. Less configurable platforms are simpler; the cost is that adapting to a new process may require code changes.

Each trade-off is not only an engineering choice but also a governance choice. The architecture makes some forms of adaptation, coordination, and oversight easier than others, whether or not the builders frame the decision politically. The framework makes these choices explicit.

4.2 Fork-Divergence Shape: A Niche-Creation Diagnostic

Fork-mining research has developed several lenses on fork–upstream divergence: evolution-pattern typologies, entropy measures of fork diversity, and longitudinal propagation studies [Zhou et al., 2020, Wang et al., 2023, Businge et al., 2022]. The premise that “merely having a large number of forks does not necessarily imply a productive and healthy OSS project” [Wang et al., 2023] and the finding that little code propagates between mainlines and divergent forks [Businge et al., 2022] both inform our reading of Consul. None of this work has been applied to civic tech, and none connects fork behaviour to SECO health dimensions explicitly.

Jansen [2014]’s niche-creation pillar reads variety (in contributor types, supported technologies, markets, and so on) and his productivity pillar reads volume of activity (lines added, spin-offs and forks, new tickets); neither pillar reads the *direction* of that activity — whether local work flows back to upstream or accumulates in isolated copies. Manikas and Hansen [2013]’s framework and Manikas [2016]’s longitudinal review treat forks descriptively and do not surface an extension-vs-silo distinction. We adapt the fork-mining literature into a population-shape diagnostic — *fork-divergence shape* — that reads the distribution of public forks across ahead-of-upstream and behind-upstream states, pairs with Jansen’s niche-creation dimension, and reads as a three-structures alignment check for civic tech platforms where forks are a relevant extension mechanism.

Operationally, each fork can be located in one of four states, defined by whether it has local commits ahead of upstream and whether it has fallen behind upstream. Table 3 names the four cells and the interpretation we attach to each.

Table 3: The four fork-divergence states. Cell colours match the quadrants of Figure 5, so the table reads as a legend for the figure.

	Behind upstream = 0	Behind upstream > 0
Ahead upstream > 0	<i>Ahead-only</i> — healthy extension; local work kept current with upstream	<i>Ahead-and-behind</i> — diverged silo; local work drifted from upstream
Ahead upstream = 0	<i>In-sync</i> — copy that mirrors upstream	<i>Behind-only</i> — dormant copy, fallen behind

The shape of the fork population, not the total count, is the diagnostic. Healthy ecosystem

extension should populate the ahead-only region: deployers keep up with upstream and add their own work, producing changes that upstream can in principle absorb. Silo formation should populate the behind-only and ahead-and-behind regions: either dormant copies fall behind, or active deployments accumulate local changes while also missing upstream changes. We apply this to Consul in Section 5.2 and find an empty ahead-only region, which we interpret as a fragmentation signal.

5 Application: Decidim and Consul

5.1 Decidim

Decidim is a modular monolith built with Ruby on Rails. The codebase at github.com/decidim/decidim contains 27 engines packaged as standalone Ruby gems at SHA `be39c244` (2026-05-12).

Software structure. The 27 engines fall into four groups: `decidim-core` as the foundational dependency, three infrastructure engines (`admin`, `api`, `system`), three shared-service engines (`comments`, `forms`, `verifications`), four participatory spaces (`processes`, `assemblies`, `conferences`, `initiatives`), and 13 components that attach to those spaces. Barandiaran et al. [2024] describe an earlier split (six spaces, twelve components); the codebase has consolidated since. Every engine except `decidim-core` declares `add_dependency "decidim-core"` in its `gemspec`, matching a hub-and-spoke pattern.

However, 13 of the 27 engines have runtime dependencies on other Decidim engines besides core: `decidim-comments` has 6 user-facing dependents and `decidim-forms` has 5, making them Layer-2 hubs of the platform (Figure 1).

The key architectural pattern is the space/component matrix: almost any component can attach to almost any space, creating an $N \times M$ combinatorial structure implemented through a polymorphic database association in `decidim-core` — a single foreign-key relationship that can point to any of the four space types, so the 13 component models do not need per-space variants. Barandiaran et al. [2024, p. 82] call this the “furniture” model: administrators mix and match components with spaces without writing code. Components are registered through a manifest API (`Decidim.register_component`); community modules use the same interface as the official ones.

To examine whether modular design is reflected in modular evolution, we applied Gall et al.’s logical coupling analysis [Gall et al., 1998] to the commit history. Of 5,517 commits (excluding mega-commits per Section 3), 64% stay within a single engine. The engine architecture provides change isolation at the repository level, though not runtime independence. The remaining cross-engine coupling is concentrated in the four participatory spaces (Jaccard up to 0.43), which share infrastructure code despite having no `gemspec` dependencies between them (Figure 2).

Organisational structure. The Decidim Free Software Association was founded on 16 February 2019 in a community assembly that approved its statutes. The Association holds the source code and trademark, transferred from Barcelona City Council in 2017. Its governance includes

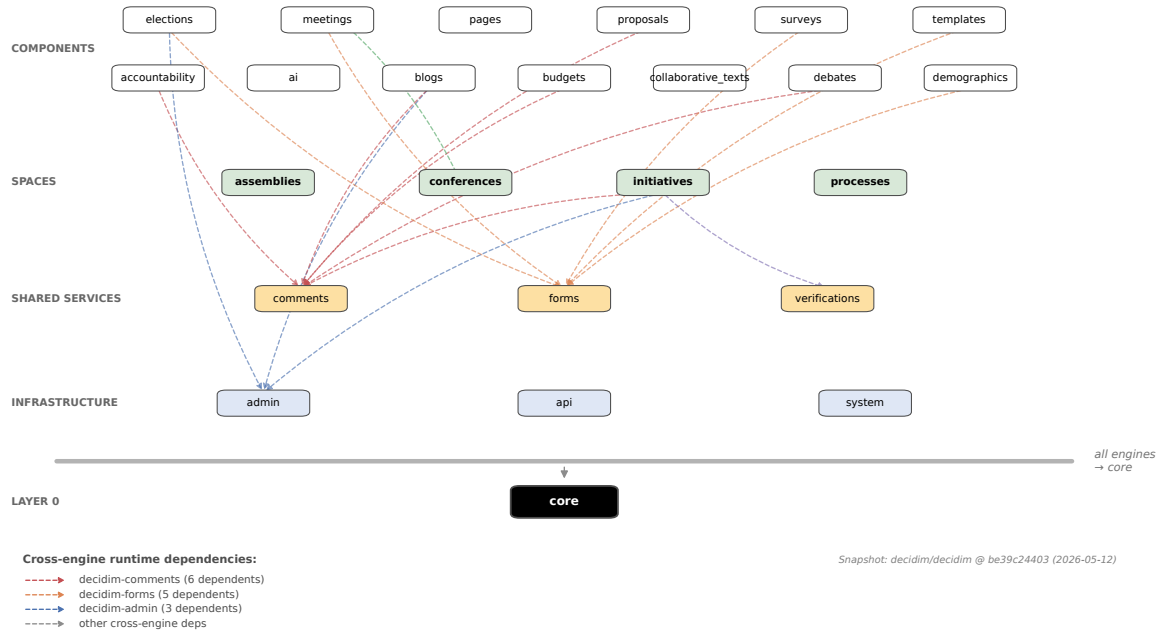


Figure 1: Decidim engine dependency graph at SHA be39c244 (2026-05-12), recovered from the 27 gemspec files. The grey bar above decidim-core stands in for the 26 universal core dependencies. The dashed arrows are cross-engine runtime dependencies that bypass core; the most-depended-on are comments (6 dependents) and forms (5). Three developer-tooling engines (design, dev, generators) are omitted.

a General Assembly as sovereign body and four standing committees: Coordination, Research, Decidim Politics, and Public Institutions. The Public Institutions Committee brings Barcelona City Council, the Generalitat de Catalunya, the Barcelona Provincial Council, Localret, and other public bodies into the coordination structure [Barandiaran et al., 2024].

MetaDecidim is the community layer: a public Decidim instance at meta.decidim.org through which both governance and product work happen. The Decidim Social Contract, binding on members, commits signatories to AGPLv3 licensing, transparency, and “equal opportunities for participatory objects” [Barandiaran et al., 2024] — the no-ranking-bias rule that surfaces in the software as a randomised default sort. The Association runs a small technical office (three employees per Barandiaran et al.) that handles release management and code review. Module ownership is distributed across Codegram engineers, Association staff, and the wider contributor base; 183 unique authors are visible in the commit log.

Business structure. The Association is funded through annual transfers from Barcelona City Council and the Generalitat de Catalunya under a formal collaboration agreement, with additional support from the Consorci Localret and the Barcelona Provincial Council [Barandiaran et al., 2024]. Around this institutional core, a network of consultancies funds itself through municipal contracts: Codegram (Barcelona), Octree (Geneva), Open Source Politics (Paris), and smaller partners. The economic loop is: municipalities procure deployments from these consultancies; the consultancies are paid for installation, customisation, hosting, and bespoke

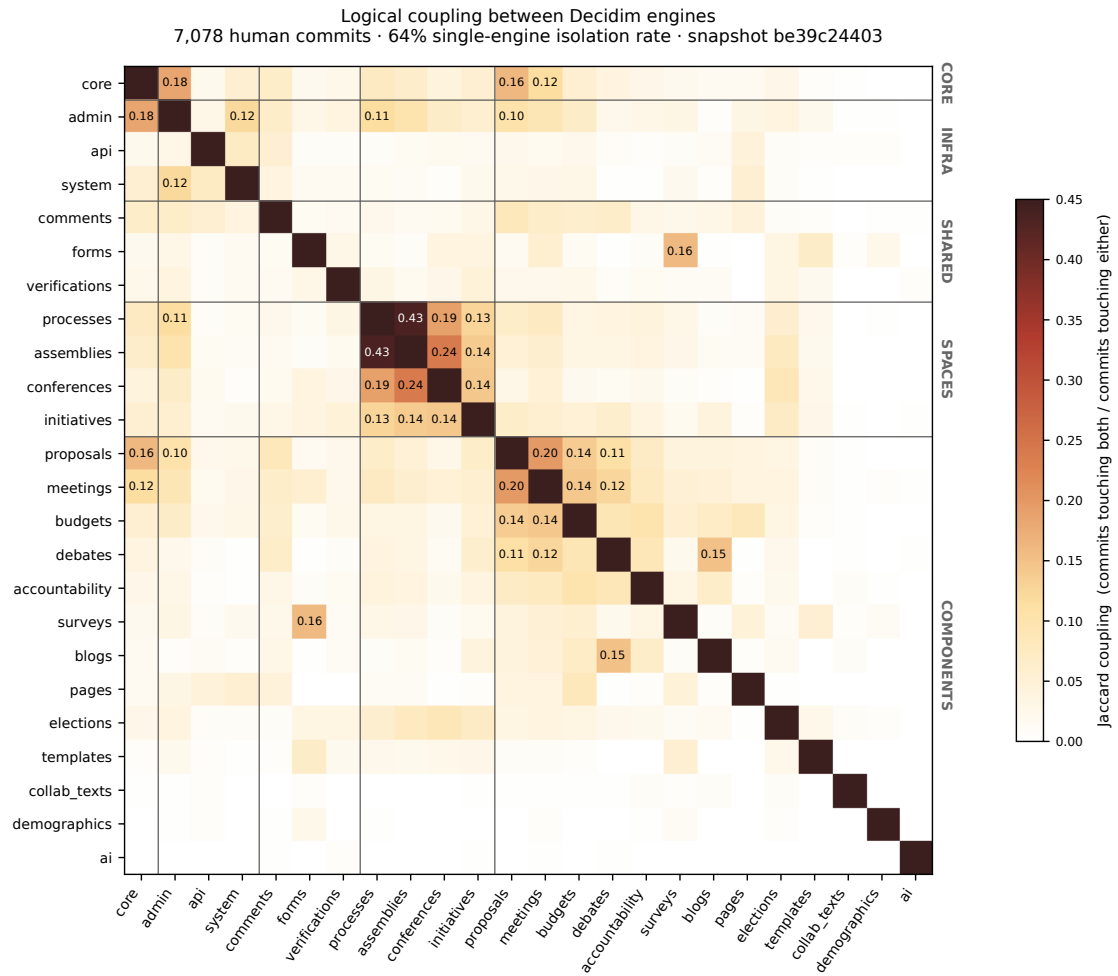


Figure 2: Logical coupling between Decidim engines, computed as Jaccard similarity over the project's commit history at SHA be39c244. The four participatory spaces (centre block) co-change despite having no gemspec dependencies between them. The shared services (comments, forms, verifications) are largely isolated — their gemspec-dependents call them as APIs without co-changing.

modules; some of the work can flow back upstream as gem-based community modules or core pull requests. Decidim documents more than 80 community-developed gems, a visible sign of this return flow.

Fork divergence. Of 452 enumerated forks of `decidim/decidim`, 444 had upstream comparison data. The distribution is concentrated in the behind-only quadrant: 399 forks (90%) have no local commits but have fallen behind upstream as Decidim ships on `develop`. Forty-five forks sit in the ahead-and-behind quadrant; none in ahead-only or in-sync. The most-divergent fork is 221 commits ahead — a Spanish-language rebrand of the documentation website, two orders of magnitude below Consul’s deepest silo (12,725 commits ahead).

The originator pattern inverts. Madrid (Consul originator) abandoned a 12,725-commit fork in 2023. Barcelona (Decidim originator, `AjuntamentdeBarcelona/decidim`) sits at zero commits ahead and forty-six behind, with a push timestamp from the day before the snapshot — the originator is tracking upstream. Codegram, the consultancy that built Decidim, also sits at zero commits ahead. `AjuntamentManacor` and other municipal forks show the same pattern. Figure 3 plots all 444 comparable forks on identical axes to Figure 5, so the contrast reads directly.

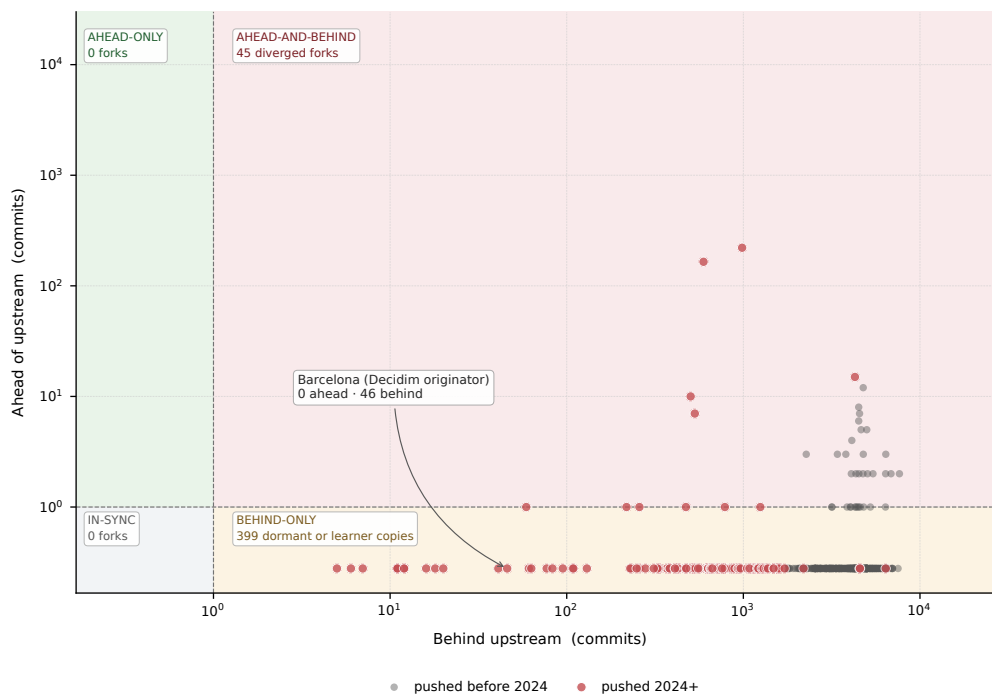


Figure 3: Decidim forks plotted by commits ahead of upstream `develop` against commits behind (snapshot 2026-05-14), on the same axes as Figure 5. The ahead-only region is empty, as for Consul. The mass sits in the behind-only quadrant (399 dormant or learner copies, 90%). The ahead-and-behind cluster is small in both count (45 forks) and magnitude (top fork at 221 commits ahead, two orders of magnitude below Consul’s Madrid). Barcelona, the Decidim originator, sits at zero ahead and forty-six behind — the inverse of Madrid’s silo. Of 452 enumerated forks, 444 had upstream comparison data.

This is not a claim that Decidim has a healthier deployment ecosystem in raw numbers. The documented installation path is `gem install decidim; decidim my_app`, which produces a new Rails application living in its own repository rather than as a fork of upstream. Most Decidim deployments are therefore invisible to fork-tree analysis. The 444 fork population

captures contributors with PRs in flight (typically single-digit ahead counts), learners reading the codebase (zero ahead, accumulating behind), and a small minority of deep-customisation deployers. The plugin path has displaced the deployment fork — not the small ahead-and-behind cluster — and that is the empirical signal.

Quality attribute profile. *Modifiability* is high. The engine architecture lets deployers add, remove, or replace components without touching core code; customisation happens through modules, not deployment forks. The 80+ community modules are evidence that this modifiability is used in practice, and all 24 user-facing engines see active maintenance in the 12 months before the snapshot — including three new ones (`ai`, `demographics`, `collaborative_texts`).

Interoperability is moderate. A GraphQL API exposes most participatory content, and the project’s integration choices (OpenStreetMap, Matomo, Jitsi) are deliberately open-source. There are no cross-platform standards for civic tech that would let Decidim compose with other platforms at the data layer, so interoperability remains less developed than modifiability or configurability in this case.

Configurability is high. The 4 spaces and 13 components produce many possible default configurations, each customisable through component-level settings without code changes. [Palacin et al. \[2024\]](#) document 25 distinct technical configurations across 31 participatory-budgeting deployments.

Transparency and auditability is high architecturally. The Decidim codebase (SHA `be39c244`) ships an immutable `ActionLog` model with a four-level visibility enum (private, admin-only, public-only, all) and an organisation scope. The `Traceable` concern adds PaperTrail-based version history to every changeable model, so per-proposal version diffs are available. Permission-gated controllers enforce admin-log access. The GraphQL API is designed to expose public participatory content while not exposing individual votes or admin actions. Sorting is deterministic and seeded for reproducibility within a session. These mechanisms make auditability a first-class architectural concern, although they do not by themselves prove that non-technical citizens can meaningfully use the audit surfaces in practice.

Three-structures alignment. The three structures align. The modular software lets external contributors ship gems without negotiating commit access to the core repository; the Association coordinates contributions and runs keystone functions such as releases, code review, and integration; the consultancy network has a business model around incremental deployment and module work; and the Public Institutions Committee channels public-body requirements back upstream. When Barcelona pays for accessibility work, that work can land in `decidim-core` and become available to other municipalities, which is the commons logic Barandiaran et al. describe. The 80+ community modules and the steady commit rate after 2019 are visible indicators of this alignment.

5.2 Consul

The original repository has since been renamed from `consul/consul` to `consuldemocracy/consuldemocracy`. Recovery is pinned to SHA `3c3b5c4a` (2026-05-11).

Software structure. Consul is a single Rails application: no gem separation, no engine isolation. Features (proposals, debates, budgets, legislation, voting, polls) share the same codebase, the same `ApplicationController`, and one `config/routes.rb`. Models live in `app/models/` with functional subdirectories but no module-level namespace boundaries enforced. All tables sit in one schema. There is no `ARCHITECTURE.md`, and the technical documentation covers installation but not architecture.

Customisation uses a custom shadow-folder pattern: to change how a budget investment works, a deployer creates a file at the same path under `app/models/custom/` that overrides the original. This reduces some file-level conflicts but does not provide module isolation. The Consul Democracy installation guide names creating a fork as the first step, making deployment forks part of the expected customisation path.

Organisational structure. The Consul Democracy Foundation was founded in 2019 as a Dutch non-profit [Linåker and Gates, 2025]. It was set up specifically because Madrid’s institutional backing for Consul collapsed after the May 2019 municipal election. The Foundation is very small: a Director, a Network Coordinator, a Tech Coordinator, and a three-person board. Crucially, the Foundation does not employ developers. Linåker and Gates [2025] report that maintenance relies on “a technical core group dominated by Spanish service suppliers dating to Consul’s early days.” The community is held together through annual ConsulCon events, a Slack workspace of 700–800 users, and a two-tier Certified Tech Partner programme. There is no Decidim-style Social Contract, no Public Institutions Committee, and no MetaDecidim-equivalent space where deployers participate in governance.

Business structure. The Foundation is funded by a patchwork: a single municipal sponsor (the City of Munich at EUR 20,000 per year), EU project grants, private donations, and revenue contributions from Certified Tech Partners [Linåker and Gates, 2025]. The Foundation does not commission core development directly; it cannot afford to. Municipalities procure Consul deployments from certified suppliers, but the architecture means much customisation happens inside the deployer’s own fork rather than as a returnable contribution. Linåker and Gates [2025] report that contributions “come largely from certified service suppliers, with fewer contributions from local governments and NGOs” and that “many development features rarely merge back to the core codebase due to architectural barriers and fragmentation across installations.”

Fork divergence. We enumerated all forks of `consuldemocracy/consuldemocracy` via the GitHub API and computed each fork’s divergence against upstream master. Of 1,030 enumerable forks, 1,026 had upstream comparison data. 746 (73%) are unmodified copies that act more like bookmarks than forks; 280 (27%) have at least one commit ahead — approximately the “around 250 deployments” figure previous reports cite [NTARI, 2025]. The distribution within those 280

is heavily skewed: most have fewer than ten commits ahead, 39 have 100 or more, and only 5 have 1,000 or more. Activity has been declining: pushes peaked in 2019 and only 14% of forks have been touched since the start of 2024 (Figure 4).

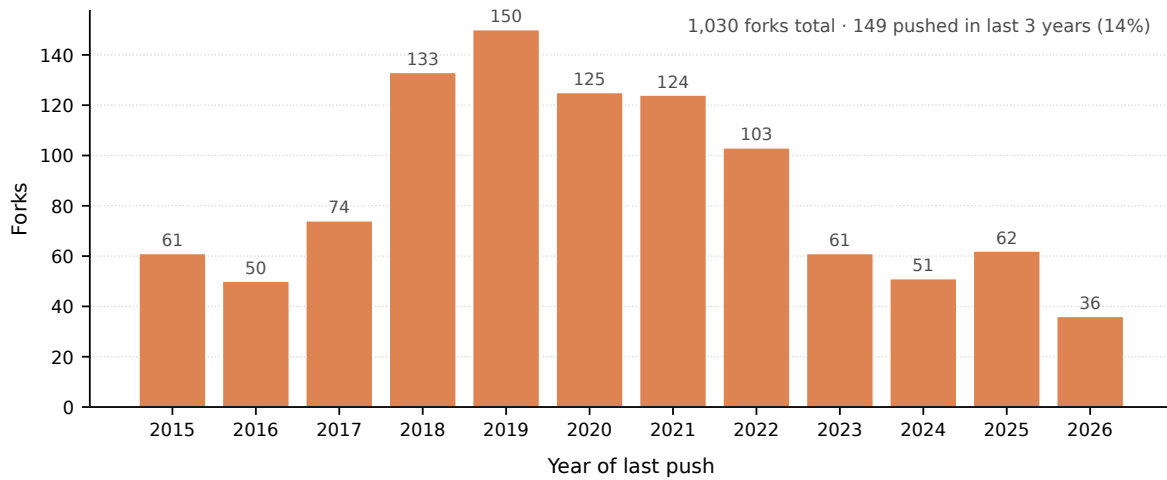


Figure 4: Last-push year distribution for the 1,030 enumerable Consul forks (snapshot 2026-05-12). Each bar counts forks whose most recent push falls in that year. Activity peaks in 2019 and declines; only 14% of forks have been touched since the start of 2024.

The most-diverged fork is `AyuntamientoMadrid/consul` — Madrid itself, the city that built Consul. It sits 12,725 commits ahead of upstream and 11,540 behind, last pushed 2023-05-23. The originating city therefore maintained a separate copy that drifted far from upstream and was eventually abandoned. This does not prove that the architecture alone caused the divergence, but it shows that even the originator ended up using the fork pattern that the platform made easiest. Plotted in Figure 5, three quadrants are populated and one is conspicuously empty: 280 ahead-and-behind silos (top-right), 745 behind-only dormant copies (bottom-right), 1 in-sync fork (bottom-left), and zero ahead-only forks (top-left). The empty top-left is the diagnostic signal — no fork keeps up with upstream while also adding local work.

Quality attribute profile. *Modifiability* is low. Customisation is expected to happen through forks and shadow-folder overrides rather than through isolated modules. Shadow folders reduce file-level conflicts but do not remove logical coupling to the upstream codebase. The 1,030 forks are the visible signature of this extension model, although not every fork represents a production deployment.

Interoperability is moderate. The public GraphQL API at `/graphql` exposes participatory content including individual vote records, but the data model is monolithic and there are no cross-platform standards.

Configurability is moderate. Several components (proposals, debates, budgets, voting) can be enabled or disabled per deployment, with phase- and quorum-level settings. The configuration is less rich than Decidim’s space/component matrix.

Transparency and auditability is moderate. Consul uses the off-the-shelf `audited` gem to track

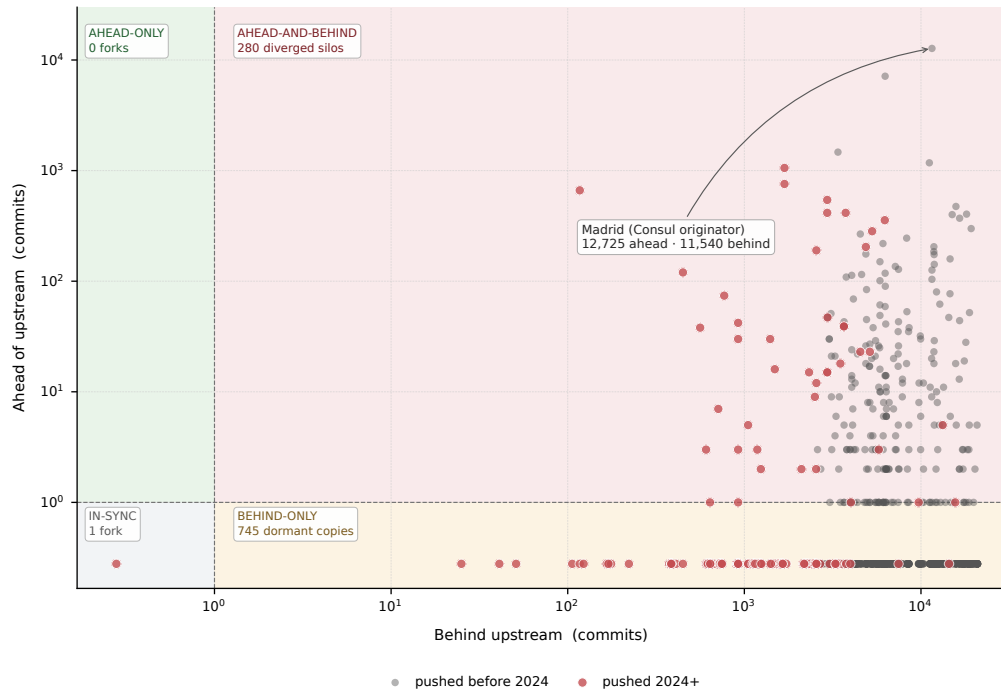


Figure 5: Consul forks plotted by commits ahead of upstream master against commits behind, on log scales (snapshot 2026-05-12). Each fork occupies one of four quadrants. The ahead-only region — where healthy contributor forks would live — is empty: no fork has added local commits while staying in sync with upstream. The mass sits in the behind-only region (745 dormant copies) and the ahead-and-behind region (280 diverged silos, including Madrid). Of 1,030 enumerated forks, 1,026 had upstream comparison data; the remaining four are excluded.

per-record changes, but the UI exposes audit history only for Budget Investments. A separate **Activity** model logs five specific moderation verbs (hide, block, restore, valueate, email). Both surfaces are admin-only. There is no Decidim-style organisation-scoped **ActionLog** with public visibility options.

Three-structures misalignment. The three structures misalign. The problem is not monolithic architecture as such: a monolith can work well when it is matched by strong central maintenance and clear contribution pathways. The problem is the fit between Consul’s monolithic extension model and its organisational and business structures. The software structure makes local adaptation easiest through forks; reintegrating that work would require a strong coordinating body with sustained development capacity; the small Foundation cannot provide that capacity; and the business structure does not reliably pay anyone to provide it. The architecture’s costs therefore land on individual deployers, who absorb them by maintaining isolated forks. The misalignment shows up in the fork data: 1,030 forks, 750 unmodified copies, and 280 modified forks with limited evidence of upstream return.

Two pieces of evidence make this concrete. First, Madrid itself ended up in the same fork pattern as other deployers, even with the resources and motivation of the originating city. Second, the post-2019 collapse: when Ahora Madrid lost the mayoralty in May 2019, the upstream commit rate dropped roughly 80% within a year. The Foundation, set up that same year, lacked core developers or significant funding; it could coordinate the ecosystem but not maintain the core. The claim is not that the monolith caused the misalignment, but that this monolithic extension model required a coordinating capacity that the organisational reality could not provide.

5.3 Cross-case Synthesis

Decidim’s contributor base is less concentrated than Consul’s, and Consul’s upstream activity collapsed in 2019. The commit history makes both patterns visible. Figure 6 shows the concentration: Decidim has 7,078 human commits across 183 unique authors with a Gini coefficient of 0.85 (top-three share 36%, top-ten 68%), against Consul’s 186 unique authors with a Gini coefficient of 0.93 (top-three 52%, top-ten 87%). These figures indicate contributor concentration in the repository, not the full distribution of influence in the wider ecosystem. Figure 7 shows the Consul cliff in 2019–2020 following Ahora Madrid’s electoral loss in May 2019 [Linåker and Gates, 2025, Barandiaran et al., 2024], alongside Decidim’s steadier plateau after 2019.

Three-structures comparison. Decidim’s structures appear aligned. The modular software affords incremental contribution; the Association coordinates it; the consultancies help sustain it. Consul’s structures appear misaligned. The monolithic extension model requires a strong coordinator that the Foundation cannot afford to be, and deployers absorb the misalignment by maintaining forks.

Trade-off positioning. The two platforms sit at opposite ends of the modifiability trade-off identified in Section 4.1.3. Decidim chose high modifiability and pays the sustainability burden of

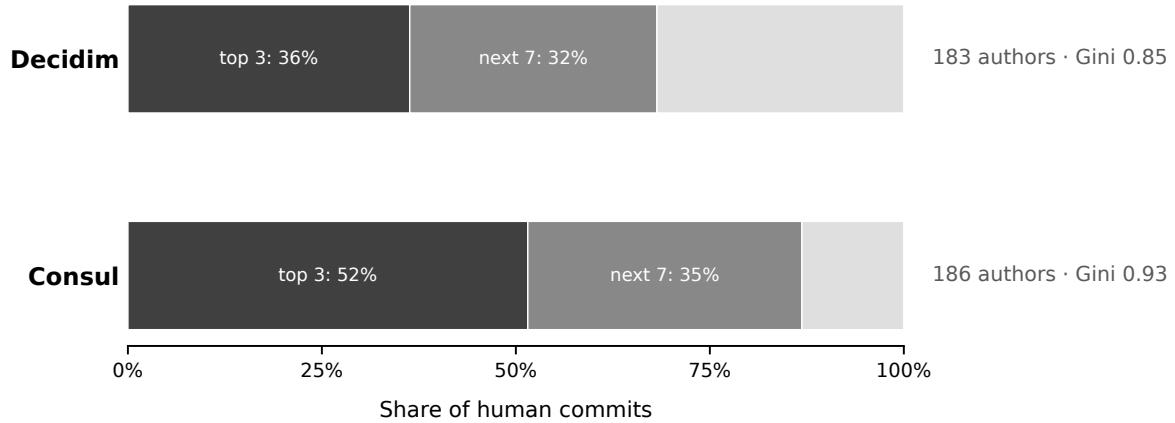


Figure 6: Share of human commits captured by the top three and top ten authors per repository. Decidim has a longer tail of unique contributors; Consul is more concentrated. Bot commits (Dependabot, Crowdin, etc.) are excluded.

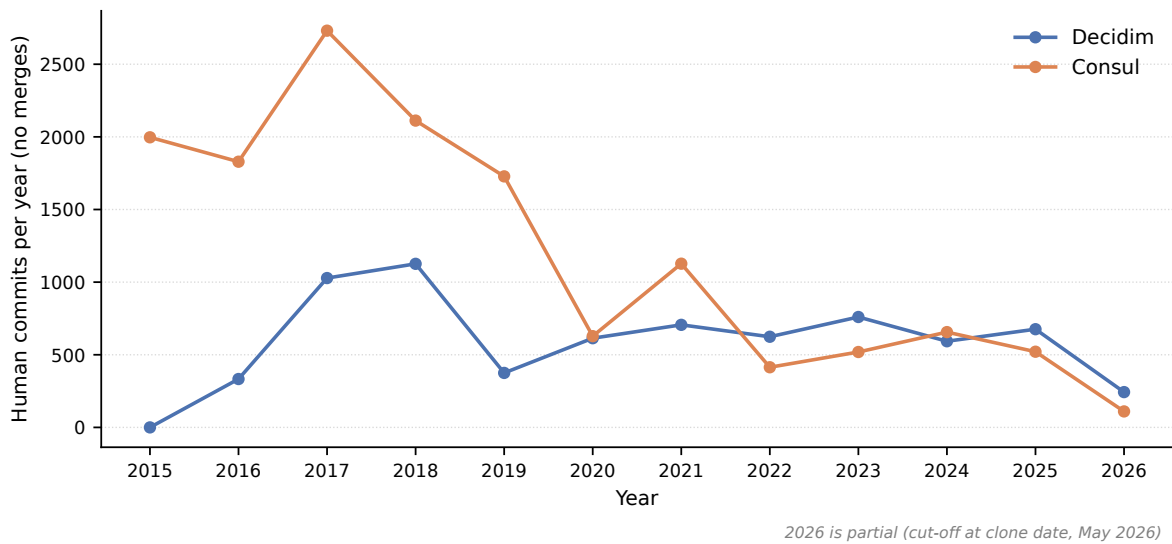


Figure 7: Human commits per year for each repository (no merges, bots excluded). The Consul cliff in 2019–2020 follows Ahora Madrid’s electoral loss in May 2019; Decidim shows a steadier plateau after 2019. The 2026 numbers are partial (cut-off at clone date).

Table 4: Three-structures analysis [Christensen et al., 2014] applied to Decidim and Consul.

Structure	Decidim	Consul
Software	Modular: 27 engines, registry pattern, shared DB, gem-based extension	Monolithic: single Rails app, shadow folder pattern, fork-based extension
Organisational	Decidim Association as keystone; MetaDecidim community; Social Contract; Public Institutions Committee	Consul Democracy Foundation (6 people, no developers); informal supplier network; weak coordination
Business	Catalan institutional funding + anchor consultancies (Codegram, Octree, Open Source Politics) + Public Institutions Committee feedback loop	EUR 20,000/yr from Munich + EU grants + supplier certification revenue; no anchor consultancy

a 27-engine codebase maintained by a small technical office. Consul chose lower modifiability and pays the cost in fork fragmentation: 280 modified forks with little visible upstream return. On interoperability, both are open-source and have GraphQL APIs but neither pursues cross-platform civic tech standards; the trade-off does not differentiate them strongly and remains under-tested in this two-case application. On configurability, Decidim’s space/component matrix is high and Consul’s per-feature settings are moderate; the cost on the Decidim side is the configuration complexity [Palacin et al. \[2024\]](#) document.

Transparency. Transparency and auditability differ sharply across the two at the architectural level. Decidim treats auditability as a first-class architectural concern through the **Action-Log/Traceable** infrastructure; Consul treats it more narrowly as a moderation oversight feature implemented via off-the-shelf gems. This differentiation supports including transparency and auditability as a primary axis of the framework, while leaving open the separate question of how usable these mechanisms are for non-technical citizens.

6 Discussion

6.1 Architecture as Governance in Practice

The Decidim/Consul comparison shows how architectural decisions participate in governance in civic technology. Decidim’s modular architecture did not only make the code flexible; it let communities and service providers shape the platform without first negotiating commit access to the core repository. An organisation can build a module, publish it as a gem, and have it work alongside the official engines. This is close to what [Schneider et al. \[2021\]](#) call “modular politics”: composable governance primitives implemented as software. The 80+ community modules are one visible artefact of this politics in action.

Consul’s monolithic extension model made a different path easier. The fork-and-shadow pattern made isolated local adaptation more straightforward than upstream contribution: deployers could either accept the platform as-is or maintain a separate fork. [Schneider \[2024\]](#) argues that platform defaults encode governance; in Consul’s case, the default extension path pulled local adaptation toward forked copies rather than shared modules, as visible at scale in the Madrid divergence (Section 5.2).

Read through [Iansiti and Levien \[2004\]](#)’s actor roles, Decidim’s Association operates as a keystone — maintaining shared infrastructure while enabling third-party modules — and the 80+ community modules are the niche-creation that follows. Consul’s Madrid team operated more like a physical dominator while institutional backing held: controlled codebase, modularisation rejected, no module mechanism for external contribution. When backing collapsed in 2019, no successor keystone emerged; deployers absorbed the vacuum by maintaining their own forks.

6.2 What SECO Research Gains

[Jansen \[2014\]](#)’s SECO health dimensions — productivity, robustness, niche creation — do not by themselves distinguish ecosystem extension from silo formation. Consul’s 2017–2019 peak

generated thousands of commits and dozens of unique contributors: by productivity measures, the ecosystem looked active. Yet the extension model pushed adopters toward separate forks, and when Ahora Madrid lost the city in 2019 the upstream commit rate dropped roughly 80%. By fork-divergence measures, the ecosystem was already vulnerable to fragmentation.

Fork-divergence shape as a niche-creation diagnostic (Section 4.2) is one concrete methodological contribution this paper makes to SECO research, adapting the fork-mining literature [Zhou et al., 2020, Wang et al., 2023, Businge et al., 2022] into a population-shape reading paired with Jansen [2014]’s niche-creation dimension for civic tech platforms. Figures 5 and 3 plot the two cases on identical axes. Consul’s quadrant pattern — 280 ahead-and-behind silos and 745 behind-only dormant copies, with the ahead-only quadrant empty — is a fragmentation signal: the deployer fork population extends away from upstream rather than around it. Decidim’s shape is structurally different: the ahead-and-behind cluster is small in both count (45 forks) and magnitude (top fork at 221 commits ahead, two orders of magnitude below Consul’s Madrid), and the originator sits at zero local divergence.

The three-structures framework of Christensen et al. [2014], applied here to civic tech, also extends the SECO toolkit. The misalignment reading of Consul’s fork fragmentation makes the structural argument concrete: not that the architecture was simply bad, but that the extension model required a coordinating capacity the organisational and business structures could not provide. Architecture and organisation diverged regardless of which way the causal arrow runs.

6.3 Implications for the Democracy Stack

International IDEA [2026] calls for a democracy stack but leaves the architectural decisions implicit. CEPS [2025] advocates for interoperable European digital public infrastructure but does not engage the modifiability-against-sustainability trade-off that shapes whether interoperable infrastructure can actually be adapted by the communities it serves. Two lessons from our cases speak to these gaps.

Modularity as a condition for pluralistic adaptation. Trade-off 1 (modifiability against sustainability burden) makes the strongest case for modular architecture in a democracy stack. The Decidim/Consul comparison shows what can happen on either side. The trade-off is real: modular maintenance is harder than maintaining a smaller central codebase. But the alternative, when local adaptation is needed and no other extension mechanism exists, is fork fragmentation that upstream cannot easily absorb. For a democracy stack that must accommodate diverse institutional contexts, modularity appears to be a central architectural condition for pluralistic adaptation. The sustainability burden should therefore be treated not as an implementation inconvenience, but as a governance cost that must be deliberately funded and coordinated.

Transparency by design, not by audit. Decidim’s transparency and auditability come from architectural decisions made early: a first-class `ActionLog`, the `Traceable` concern wired into changeable models, and a GraphQL API designed to expose public participatory content while protecting sensitive data. Consul’s transparency mechanisms are narrower and implemented

mainly through off-the-shelf audit and activity-tracking features. For a democracy stack that has to remain auditable across deployments and over time, transparency is difficult to add as a superficial feature after launch; it has to be treated as an architectural property. The cost — audit infrastructure across models, permissions, visibility levels, and APIs — is real, but without it the legitimacy of participatory outcomes is harder to verify.

6.4 Limitations

The two-platform comparison is the framework’s first worked application, not a validation across the field. Several limitations bound what we can claim.

Causality. The architecture-as-governance reading is correlational. Barcelona and Madrid had different governance cultures and political trajectories; Madrid’s Ahora Madrid government lost power in 2019, hurting Consul’s institutional backing. Separating what the architecture caused from what politics caused would need interview research we did not do. The architecture and the organisational reality diverged in opposite directions, and the misalignment matters regardless of which way the causal arrow runs.

Single coder. The architectural and ecosystem judgements were made by one author; a second coder could disagree on edge cases (which engine counts as “shared service”, which fork counts as “substantively modified”). Where possible we triangulated against published characterisations — Linåker and Gates [2025] on Consul’s organisational structure, Barandiaran et al. [2024] on Decidim’s governance, the OSOR and NTARI reports on Consul deployments — and noted disagreements explicitly.

Anglophone bias. Almost all our sources are English; Decidim has substantial Spanish- and Catalan-language documentation, and the Consul fork landscape is heavily Spanish, Catalan, and Latin American. We likely missed material that would refine our reading.

Selection on the dependent variable. We chose two platforms that exist; the civic tech graveyard — platforms that never reached production or failed invisibly — is invisible to this study. The trade-offs we identify are conditional on survival.

Architectural recovery limits. Architectural recovery captures dependency structure and extension mechanisms, not runtime behaviour. Evolutionary metrics use email-based author identity, which conflates authors with multiple emails. Fork analysis covers public forks only and may miss private production deployments. Each of these is a real limit on what the methods can see.

7 Conclusion

Decidim and Consul started from the same political moment in 2015–2016 and ended up as different ecosystems. Decidim’s 27 engines sit on top of a coordinating Association and consultancies that fund incremental contribution; 80+ community modules and a Barcelona originator tracking upstream at zero local commits are its signs. Consul’s monolithic codebase sits on top of a six-person Foundation with no developers and EUR 20,000 of municipal funding;

280 ahead-and-behind silos and Madrid’s abandoned 12,725-commit fork are its signs. The structural fit, not just the software, decides what the ecosystem becomes.

Four architecture decisions did most of the work. *Extension model*: gem-based engines let outsiders ship code without core access; fork-and-shadow made local adaptation easier than upstream return. *Modularity granularity*: a 27-engine codebase produced procedural pluralism without code changes; a single Rails app pushed customisation into the deployer’s source tree. *Audit primitives as first-class architecture*: **ActionLog** and **Traceable** bought public-facing auditability for Decidim; an off-the-shelf **audited** gem bought admin-side moderation oversight for Consul. *Coordinating capacity*: Decidim’s modular structure presumes a keystone and the Association supplies it; Consul’s monolithic extension model presumes the same capacity but the small Foundation cannot fund it.

The EU policy frame — DSA, eIDAS, IDEA’s democracy stack, CEPS — is mostly silent on the architectural *how*. Modularity is a condition for pluralistic adaptation, not a nice-to-have; the sustainability burden is a governance cost, not an engineering inconvenience. Transparency that has to survive across deployments has to be wired into the architecture, not bolted on. The harder question, which the architecture alone cannot answer, is who plays the keystone role when no single municipality or association can carry it across a European democracy stack.

Future research. Two directions invite follow-on work. First, extending the framework to a wider set of platforms: CitizenLab for sovereignty under commercial SaaS, Loomio for worker-cooperative governance, and a federated platform for security and identity. Second, stakeholder interviews with Decidim deployers, Consul fork maintainers, and Association members would complement the repository-mining done here and address the causal limitations in Section 6.4.

Use of Generative AI

Tool used: Claude (Anthropic; Claude Sonnet 4.6 and Claude Opus 4.7), accessed via the Claude Code command-line interface.

Use in the project: Generative AI was used as an assistant during the research and writing process. It supported literature discovery, outlining and structural decisions, language refinement and copy editing, exploratory web research, and the drafting and debugging of Python and matplotlib scripts used to collect, process, and visualise repository data. The scripts and paper source are publicly available alongside the paper on GitHub (see Section 3, “Data sources”).

Author contribution and validation: The author developed the research question, case selection, architectural framing, quality-attribute analysis, interpretation of findings, prose, and final argumentative structure. AI was used as a research and writing assistant — supporting refinement of drafts, language editing, code review, and exploration of alternative framings — with all AI-generated content reviewed, revised, tested, and validated by the author against primary sources, public documentation, and repository data. The analytical claims, figures, and conclusions presented in the paper are the author’s own responsibility.

Data, copyright, and confidentiality: No confidential material or personal data was provided to the AI tool. The work was based on the author’s notes, public documentation, public repositories, and publicly available sources.

All submitted text, figures, code, and analysis were reviewed, edited, and approved by the author.

References

- Xabier E. Barandiaran, Antonio Calleja-López, Arnau Monterde, and Daniel Romero. *Decidim, a Technopolitical Network for Participatory Democracy*. Springer, 2024.
- Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 4th edition, 2021.
- John Businge, Moses Openja, Sarah Nadi, and Thorsten Berger. Reuse and maintenance practices among divergent forks in three software ecosystems. *Empirical Software Engineering*, 27(2):54, 2022. doi: 10.1007/s10664-021-10078-2.
- CEPS. Building the European digital public infrastructure: Rationale, options, and roadmap. Technical report, Centre for European Policy Studies, 2025.
- Henrik Bærbak Christensen, Klaus Marius Hansen, Morten Kyng, and Konstantinos Manikas. Analysis and design of software ecosystem architectures — towards the 4S teahouse. *Information and Software Technology*, 56(11):1476–1492, 2014.
- Codegram. Building Decidim: Architecture overview. <https://www.codegram.com/blog/building-decidim-architecture-overview/>, 2024. Accessed 2026.
- Laura DeNardis. *The Global War for Internet Governance*. Yale University Press, 2014.
- Stéphane Ducasse and Damien Pollet. Software architecture reconstruction: A process-oriented taxonomy. *IEEE Transactions on Software Engineering*, 35(4):573–591, 2009.
- European Union. Regulation (eu) 2022/2065 of the european parliament and of the council of 19 october 2022 on a single market for digital services and amending directive 2000/31/ec (digital services act). Official Journal of the European Union, L 277, 27.10.2022, pp. 1–102, 2022. URL <https://eur-lex.europa.eu/eli/reg/2022/2065/oj/eng>. Accessed May 2026.
- European Union. Regulation (eu) 2024/1183 of the european parliament and of the council of 11 april 2024 amending regulation (eu) no 910/2014 as regards establishing the european digital identity framework. Official Journal of the European Union, L, 2024/1183, 30.4.2024, 2024. URL <https://eur-lex.europa.eu/eli/reg/2024/1183/oj/eng>. Accessed May 2026.
- Harald Gall, Karin Hajek, and Mehdi Jazayeri. Detection of logical coupling based on product release history. In *Proceedings of the International Conference on Software Maintenance (ICSM 1998)*, pages 190–198. IEEE Computer Society, 1998.
- Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. A comparative analysis of software

- architecture recovery techniques. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 486–496. IEEE, 2013.
- Ahmed E. Hassan. The road ahead for mining software repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE, 2008.
- Marco Iansiti and Roy Levien. Strategy as ecology. *Harvard Business Review*, 82(3):68–81, 2004.
- International IDEA. The case for a democracy stack: Rethinking digital public infrastructure for democratic societies. Technical report, International Institute for Democracy and Electoral Assistance, 2026. URL <https://www.idea.int/publications/catalogue/case-democracy-stack>. Accessed May 2026.
- Slinger Jansen. Measuring the health of open source software ecosystems: Beyond the scope of project health. *Information and Software Technology*, 56(11):1508–1519, 2014.
- Antti Knutas, Timo Hynninen, and Maija Hujala. Local solutions with global reach: Can civic tech benefit from open source software ecosystem practises? In *Proceedings of the 12th International Workshop on Software Ecosystems (IWSECO 2020)*, 2020.
- Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006.
- Johan Linåker and Nicholas Gates. Case study of Consul Democracy. Technical report, OSOR/Interoperable Europe, 2025.
- Konstantinos Manikas. Revisiting software ecosystems research: A longitudinal literature study. *Journal of Systems and Software*, 117:84–103, 2016.
- Konstantinos Manikas and Klaus Marius Hansen. Software ecosystems — a systematic literature review. *Journal of Systems and Software*, 86(5):1294–1306, 2013.
- NTARI. Consul Democracy: A report on forks, distribution, and modifications. Technical report, NTARI, 2025. URL <https://www.ntari.org/post/consul-democracy-a-report-on-forks-distribution-and-modifications>. Accessed May 2026.
- Victoria Palacin, Sean McDonald, Pablo Aragón, and Matti Nelimarkka. Configurations of digital participatory budgeting. *ACM Transactions on Computer-Human Interaction*, 31(2), 2024.
- Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.
- Nathan Schneider. *Governable Spaces: Democratic Design for Online Life*. University of California Press, 2024.
- Nathan Schneider, Primavera De Filippi, Seth Frey, Joshua Z. Tan, and Amy X. Zhang. Modular politics: Toward a governance layer for online communities. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW1), 2021.

- Liang Wang, Zhiwen Zheng, Baihui Sang, Xiangchen Wu, Jierui Zhang, and Xianping Tao. Fork entropy: Assessing the diversity of open source software projects' forks. *arXiv preprint arXiv:2205.09931*, 2023. Version 2, 19 September 2023.
- Langdon Winner. Do artifacts have politics? *Daedalus*, 109(1):121–136, 1980.
- Amy X. Zhang, Grant Hugh, and Michael S. Bernstein. PolicyKit: Building governance in online communities. In *UIST '20*, pages 365–378, 2020.
- Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. How has forking changed in the last 20 years? A study of hard forks on GitHub. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE '20)*, Seoul, Republic of Korea, 2020. ACM. doi: 10.1145/3377811.3380412.